

Práctica 4: Módulos Odoo Modelo y Vista.

Andrea Sofía Pais Dos Santos

December 11, 2025

1 Índice

2. Introducción	página 3
3. Actividad 01 - Lista Tareas	página 4
• Nueva vista para ver las tareas en formato "Kanban"	
• Vista visualización "Calendario" según fecha	
• Errores encontrados	
4. Actividad 02 - Biblioteca de Cómics	página 8
• Gestionar socios y atributos	
Errores encontrados	
• Gestión de préstamos	
• Restricciones	
Errores encontrados	
5. Actividad 03 - Módulo Hospital	página 18
• Modelo Paciente	
• Modelo Médico	
• Modelo Consulta	
• Errores encontrados	
6. Actividad 04 - Módulo Colegio	página 28
• Crear 4 modelos (Ciclo Formativo, Módulo, Alumno y Profesor)	
• Implementar los modelos, relaciones y vistas necesarias	
• Configuración de seguridad	
• Errores encontrados	
7. Conclusiones	página 44

2 Introducción

El objetivo de la práctica es crear y modificar módulos para seguir adquiriendo conceptos y ser capaces de crear módulos por nuestra cuenta. Nos vamos a basar en ejemplos del 01 al 06 del repositorio que nos proporcionan en el enunciado: [repositorio de ejemplo](#). Primero vamos a tener en cuenta que el primero ejercicio lo vamos a realizar en base al módulo de la práctica anterior de la "lista de tareas" por lo que vamos a seguir usando la práctica anterior.

Hay que tener en cuenta que para ir viendo si funciona los módulos y sus respectivas vistas estén correctos tenemos que levantar el contenedor en docker, iniciar en Odoo con nuestros datos, actualizar la lista de aplicaciones y realizar lo que necesitemos: activar módulo, actualizar módulo, etc.

3 Actividad 01 - Lista Tareas

El objetivo de la primera actividad es modificar el módulo que creamos en la anterior práctica añadiéndole dos vistas nuevas. La primera una vista Kanban y la segunda una vista que visualiza un Calendario.

3.1 Nueva vista en formato Kanban

- Creamos un nuevo archivo xml "view kanban" que va a tener la siguiente estructura pero un nivel básico para ir entendiendo poco a poco:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Vista Kanban para la lista de tareas -->
4     <record id="lista_tareas_kanban_view" model="ir.ui.view">
5         <field name="name">lista_tarea.kanban.view</field>
6         <field name="model">lista_tareas.lista_tareas</field>
7         <field name="arch" type="xml">
8             <!-- Campos necesarios agrupados por el estado que es lo que hace mas
9              agil el estilo kanban-->
10            <kanban default_group_by="realizada">
11                <field name="tarea"/>
12                <field name="categoria"/>
13                <field name="prioridad"/>
14                <field name="urgente"/>
15                <field name="realizada"/>
16
17            <!-- Vista de cada tarjeta en la vista Kanban -->
18            <templates>
19                <t t-name="kanban-box">
20                    <div class="oe_kanban_card oe_kanban_global_click">
21                        <!-- Nombre de la tarea -->
22                        <strong><field name="tarea"/></strong>
23                        <div>
24                            <!-- Mostrar la etapa (estado) de la tarea -->
25                            <span>Prioridad: <field name="prioridad" /></span>
26                            <br/>
27                        </div>
28                    </div>
29                </t>
30            </templates>
31        </field>
32    </record>
33 </odoo>
```

```

27         </t>
28     </templates>
29 </kanban>
30 </field>
31 </record>
32 </odoo>

```

- Ponemos los campos que tenemos en el modelo del módulo, luego vamos a tener que añadir el del tiempo para el calendario.
- Necesitamos que se vea en otra vista por lo que en la vista principal tenemos que añadir a las acciones kanban.

```

1 <field name="view_mode">list,form,kanban</field>

```

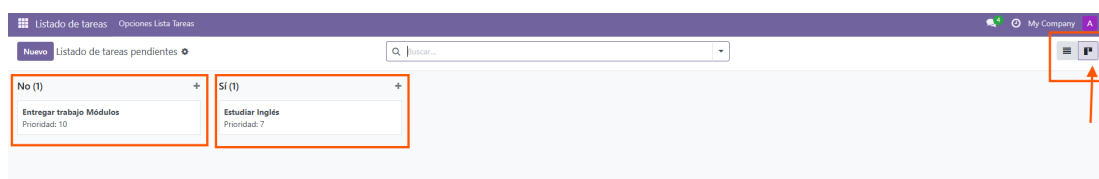
- Para que el modelo tenga en cuenta al la vista que creamos tenemos que añadir la vista al archivo manifest.

```

1 # -*- coding: utf-8 -*-
2 {
3     'name': "Lista de tareas",
4     'summary': '"Sencilla Lista de tareas"',
5     'description': '"Sencilla lista de tareas utilizadas para crear un nuevo
6         modulo con un nuevo modelo de datos"',
7     'author': "Andrea Sofia Pais Dos Santos",
8     'application': True,
9     'category': 'Productivity',
10    'version': '0.1',
11    'depends': ['base'],
12    'data': [
13        'security/ir.model.access.csv',
14        'views/views.xml',
15        'views/view_kanban.xml' # -*- hay que poner las vistas que vamos creando
16    ],
17 }

```

- Al realizar los cambios lo que tenemos que hacer es actualizar la aplicación que habíamos creado en la otra práctica. Al entrar a la aplicación vemos la vista que principal y ahora vemos una que hay una nueva vista que nos enseña las tareas en formato tarjeta. Las podríamos hacer más bonitas con css, este sería el estilo básico de kanban. Se dividen en dos columnas los que están realizadas y las que no.



3.2 Vista visualización "Calendario" según fecha

- Hay que seguir más o menos los mismos pasos que para hacer la vista kanban. Creamos un archivo para el Calendario, pero lo primero es añadir un atributo nuevo a la vista principal que va a ser la fecha asignada.
- En el archivo models vamos a añadir el campo fecha asignada de las tareas que se va a crear.

```
1 from odoo import models, fields, api
2 #Definimos el modelo de datos
3 class lista_tareas(models.Model):
4     #Nombre y descripcion del modelo de datos
5     _name = 'lista_tareas'
6     _description = 'lista_tareas'
7     #Elementos de cada fila del modelo de datos
8     #Los tipos de datos a usar en el ORM son
9     tarea = fields.Char(string="Tarea")
10    prioridad = fields.Integer(string="Prioridad")
11    urgente = fields.Boolean(compute="_value_urgente", store=True)
12    realizada = fields.Boolean(string="Realizada")
13    categoria = fields.Selection([
14        ("estudio", "Estudio"),
15        ("trabajo", "Trabajo"),
16        ("ocio", "Ocio"),
17        ("cita", "Citas")
18    ], string="Categoria", default="cita")
19    date_deadline= fields.Date(string="Fecha", default=fields.Date.today) #
20        Ponemos el campo fecha que necesitamos para el calendario
21    @api.depends('prioridad')
22    def _value_urgente(self):
23        #Para cada registro
24        for record in self:
25            #Si la prioridad es mayor que 10, se considera urgente, en otro caso no
26            lo es
27            if record.prioridad>10:
28                record.urgente = True
29            else:
30                record.urgente = False
```

- Luego en la vista principal añadimos el atributo fecha asignada.

```
1 # ... trozo del archivo
2 <record model="ir.ui.view" id="lista_tareas_form">
3     <field name="name">lista_tareas_form</field>
4     <field name="model">lista_tareas</field>
5     <field name="arch" type="xml">
6         <form>
7             <sheet>
8                 <group>
9                     <group string="La informacion principal">
10                         <field name="tarea" required="1"/>
11                         <field name="categoria"/>
12                         <field name="prioridad"/>
13                         <field name="date_deadline"/> #Ponemos el atributo fecha nuevo
14                     </group>
```

```

15         <group string="Estado">
16             <field name="realizada"/>
17             <field name="urgente"/>
18         </group>
19     </group>
20 </sheet>
21 </form>
22 </field>
23 </record>
24 ...

```

- Ahora en el archivo que creamos para la vista Calendario vamos a tener la siguiente estructura:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <record id="lista_tareas_calendar_view" model="ir.ui.view">
4         <field name="name">lista_tareas.calendar.view</field>
5         <field name="model">lista_tareas</field>
6         <field name="arch" type="xml">
7             <!-- Estructura de la vista calendario-->
8             <calendar string="Calendario de Tareas"
9                 date_start="date_deadline"
10                 date_stop="date_deadline"
11                 color="categoria"
12                 mode="month">
13                 <!-- Campos que se muestran -->
14                 <field name="tarea"/>
15                 <field name="prioridad"/>
16                 <field name="date_deadline"/>
17             </calendar>
18         </field>
19     </record>
20 </odoo>

```

- En el manifest hay que añadir la vista Calendario.

```

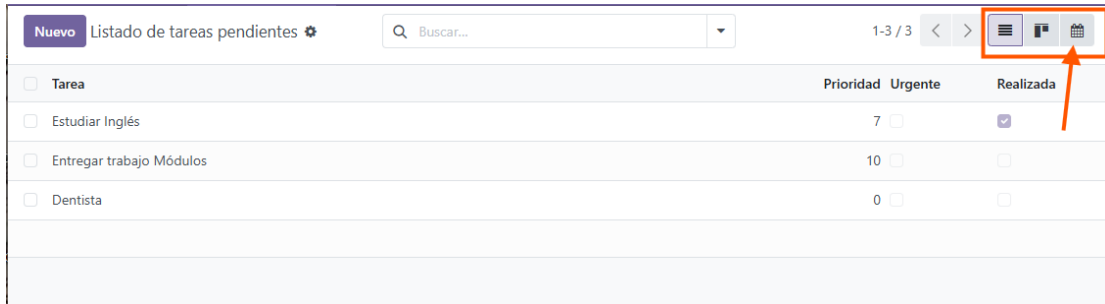
1 # -*- coding: utf-8 -*-
2 {
3     'name': "Lista de tareas",
4     'summary': """"Sencilla Lista de tareas""",
5     'description': """"Sencilla lista de tareas utilizadas para crear un nuevo
6         modulo con un nuevo modelo de datos""",
7     'author': "Andrea Sofia Pais Dos Santos",
8     'application': True,
9     'category': 'Productivity',
10    'version': '0.1',
11    'depends': ['base'],
12    'data': [
13        'security/ir.model.access.csv',
14        'views/views.xml',
15        'views/view_kanban.xml', # -*- hay que poner las vistas que vamos
16            creando -*-
17        'views/view_calendario.xml' # -*- y ahora la vista para el calendario -*-
18    ],
19 }

```

- Necesitamos que se vea en otra vista por lo que en la vista principal tenemos que añadir a las acciones el calendario.

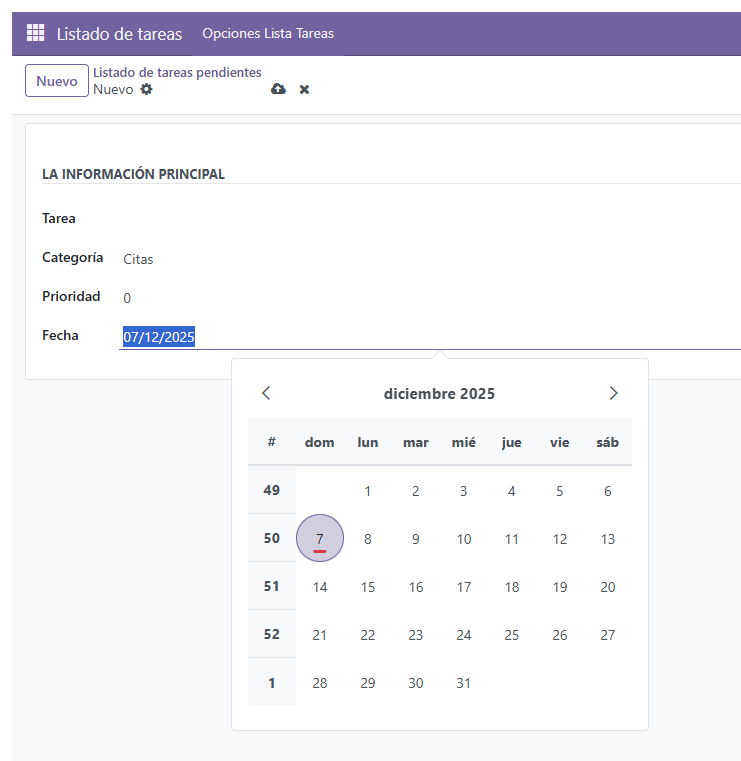
1 `<field name="view_mode">list,form,kanban,calendar</field>`

- Al tener todos los archivos bien, actualizamos el módulo vemos que tenemos una nueva vista que nos abre a un calendario mensual que almacena las tareas que vamos añadiendo.



Tarea	Prioridad	Urgente	Realizada
☐ Estudiar Inglés	7	☐	☒
☐ Entregar trabajo Módulos	10	☐	☐
☐ Dentista	0	☐	☐

- Por ejemplo, vamos a crear una nueva tarea y ahora nos da la opción de seleccionar una fecha en la que ocurrirá la tarea.



LA INFORMACIÓN PRINCIPAL

Tarea

Categoría Citas

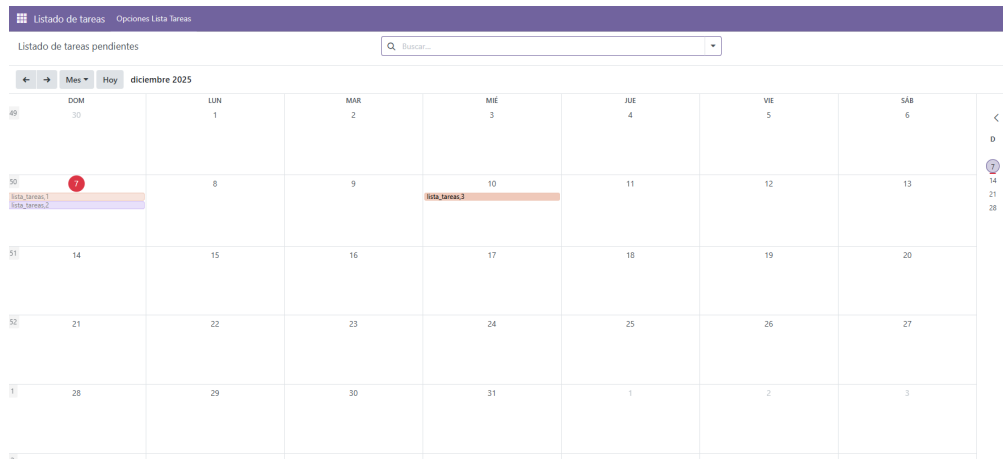
Prioridad 0

Fecha 07/12/2025

< diciembre 2025 >

#	dom	lun	mar	mié	jue	vie	sáb
49		1	2	3	4	5	6
50	7	8	9	10	11	12	13
51	14	15	16	17	18	19	20
52	21	22	23	24	25	26	27
1	28	29	30	31			

- Ahora, accedemos al calendario y vemos las tareas



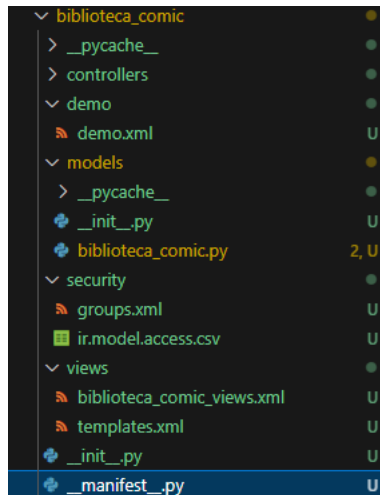
3.3 Errores encontrados

- Tuve un error que no me dejaba actualizar la aplicación de la lista de tareas y era porque me faltaba añadir la lista al manifest.
- Un error que tuve al actualizar la aplicación me detectó un error en el nombre del módulo. En esta versión de Odoo, el nombre del modelo debe de ser simplemente el nombre del módulo, no duplicado. Y lo tuve que cambiar en todos los archivos.

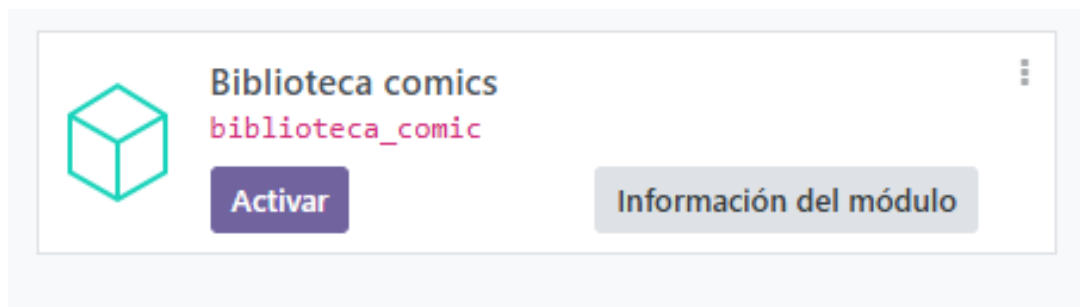


4 Actividad 02 - Biblioteca de Cómic

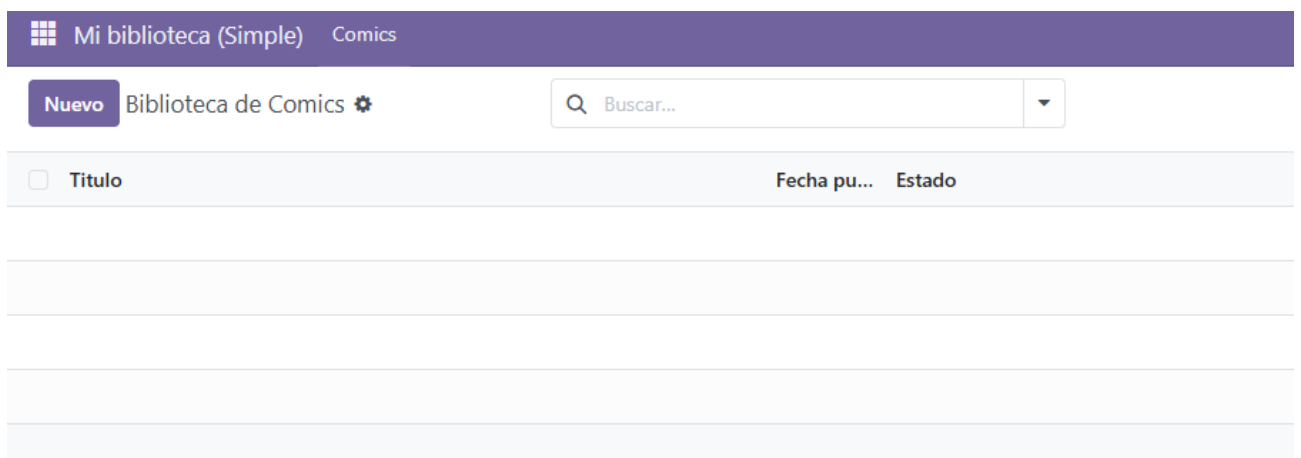
Para este ejercicio vamos a modificar a partir de un módulo base de una biblioteca que fue aportado para que implementemos como base para la práctica y hacer las modificaciones que se nos pide adicional. Como sería gestionar socios. ejemplares para socios... Para empezar vamos a crear la carpeta con el módulo con sus respectivas carpetas y archivos.



Añadimos el contenido de los archivos para crear la base de la librería basándonos en los ejercicios de muestra que están en el repositorio proporcionados y entender el contenido que estamos realizando. Luego reiniciamos el contenedor o con actualizar la lista de aplicaciones bastaría, buscamos el nuevo módulo y lo activamos.



Si todo está correcto, podemos acceder al módulo en aplicaciones instaladas.



4.1 Gestión de socios

Primero vamos a crear un nuevo modelo dentro de nuestra carpeta models, en este caso lo llamaremos "biblioteca socio.py". Y definiremos los campos que se deben de registrar de cada socio. Podemos usar como base el modelo principal manteniendo la base y añadiendo "nombre del socio", apellido y id para que sea único cada socio.

```

1  # -*- coding: utf-8 -*-
2  from odoo import models, fields, api
3
4  #Definimos modelo Biblioteca socio
5  class BibliotecaComic(models.Model):
6
7      #Nombre y descripcion del modelo
8      _name = 'biblioteca.socio'
9      #Hereda de "base.archive"
10     _inherit = ['base.archive']
11     _description = 'Socios de la biblioteca'
12
13     #ATRIBUTOS
14
15     nombre = fields.Char('Nombre', required=True)
16     apellido = fields.Char('Apellido', required=True)
17     identificacion = fields.Char('Identificador', required=True)
18     #Indicamos que atributo sera el que se usara para mostrar nombre.
19     #Aqui indicamos que se use el atributo "nombre"
20     _rec_name = 'nombre'
21
22     _sql_constraints = [
23         ('identificador_uniq', 'UNIQUE(identificador)', 'El identificador ya existe')
24     ]

```

Ahora en el archivo "init" de la carpeta models, importamos el nuevo archivo que acabamos de crear.

```

1  # -*- coding: utf-8 -*-
2
3  from . import biblioteca_comic
4  from . import biblioteca_socio

```

Después tenemos que crear la vista del nuevo panel para crear nuevos socios y poder modificarlos. Se va a crear un formulario con los datos a introducir, un listado y una opción para poder buscar los socios.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3
4      <!-- Definimos como mostramos la vista en el modelo -->
5      <record id='biblioteca_comic_action_socios' model='ir.actions.act_window'>
6          <field name="name">Socios</field>
7          <!-- Indicamos a que modelo aplica -->
8          <field name="res_model">biblioteca.socio</field>
9          <!-- Indicamos que los socios pueden verse en list para el listado,
10             y en form para la creacion/edicion -->
11          <field name="view_mode">list,form</field>
12      </record>
13
14      <!-- Definicion de la Vista formulario -->
15      <record id="biblioteca_socio_view_form" model="ir.ui.view">
16          <field name="name">Formulario de Comic de la biblioteca</field>
17          <field name="model">biblioteca.socio</field>

```

```

18     <field name="arch" type="xml">
19         <form>
20             <!-- Creamos un boton "archivar".
21                  Este metodo archivar cambie el estado del atributo del modelo "
22                      activo"
23             -->
24             <header>
25                 <button type="object" name="archivar" string="Archivar Socios"/>
26             </header>
27             <group>
28                 <group>
29                     <!-- Atributos -->
30                     <field name="nombre"/>
31                     <field name="apellido"/>
32                     <field name="identificacion"/>
33                 </group>
34             </group>
35         </form>
36     </field>
37 </record>
38
39 <!-- Definir de la vista list -->
40 <record id="biblioteca_socio_view_list" model="ir.ui.view">
41     <field name="name">Lista de Socios de la Biblioteca</field>
42     <field name="model">biblioteca.socio</field>
43     <field name="arch" type="xml">
44         <list>
45             <field name="nombre"/>
46             <field name="apellido"/>
47             <field name="identificacion"/>
48         </list>
49     </field>
50 </record>
51
52 <!-- Definir de la vista busqueda-->
53 <record id="biblioteca_socio_view_search" model="ir.ui.view">
54     <field name="name">Busqueda de los Socios</field>
55     <field name="model">biblioteca.socio</field>
56     <field name="arch" type="xml">
57         <search>
58             <field name="nombre"/>
59             <field name="apellido"/>
60             <field name="identificacion"/>
61         </search>
62     </field>
63 </record>
64 </odoo>

```

La nueva vista que creamos la tenemos que añadir en nuestro archivo "manifest"

```

1 # -*- coding: utf-8 -*-
2 {
3     'name': "Biblioteca comics",
4     'summary': '"Biblioteca de comics"',
5     'description': '"Gestion de ejemplares y socios"',
6     'author': "Andrea Sofia Pais Dos Santos",

```

```

7  'application': True,
8  'category': 'Productivity',
9  'version': '0.1',
10 'depends': ['base'],
11 'data': [
12     'security/ir.model.access.csv',
13     'security/groups.xml',
14     'views/biblioteca_socio.xml', # ponemos las nuevas vistas que vamos creando
15     'views/biblioteca_comic_views.xml',
16 ],
17 }

```

Para que podamos navegar a la vista que hemos creado, en la vista principal vamos a añadir al menú principal un nuevo apartado. Por lo que vamos a tener que modificar el archivo de vista principal "biblioteca comic view.xml" y añadir esta línea:

```

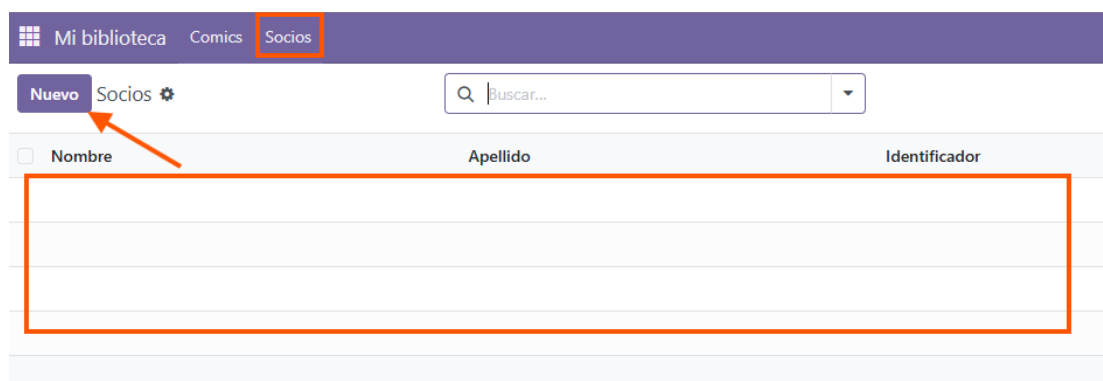
1  # .....
2  <menuitem name="Mi biblioteca" id="biblioteca_base_menu" />
3  <menuitem name="Comics" id="biblioteca_comic_menu" parent="biblioteca_base_menu"
4    action="biblioteca_comic_action"/>
5  <!-- Creamos un nuevo apartado para la vista de Socios-->
6  <menuitem name="Socios" id="biblioteca_comic_menu_socio" parent="
7    biblioteca_base_menu" action="biblioteca_comic_actions_socios"/>
8  # .....

```

Si todo está correcto, actualizamos el módulo y nos aparece un nuevo apartado en el menú principal.



Ahora ya podemos ver el nuevo apartado "Socios".



Dentro de la sección "Socios" podemos crear los socios que queramos con el nombre, apellido y identificación.

Mi biblioteca Comics Socios
 Nuevo Socios Nuevo
 Archivar Socios
 Nombre Andrea Sofía
 Apellido Pais Dos Santos
 Identificador 123Andrea

Al estar creado ese socio pasa a estar en la lista de socios en la que podemos ver sus datos.

☐	Nombre	Apellido	Identificador
☐	Andrea Sofía	Pais Dos Santos	123Andrea

4.1.1 Errores encontrados

- Un error que tuve fue no añadir al archivo `ir.model.access.csv` el nuevo modelo que creamos.

```

1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_biblioteca_comic,biblioteca.comic.access,model_biblioteca_comic,base.
   group_user,1,1,1,1
3 access_biblioteca_socio,biblioteca.socio.access,model_biblioteca_socio,base.
   group_user,1,1,1,1

```

- Y el resto de errores eran conflictos que había de definición de los atributos mal escritos sin darme cuenta.

¡Vaya!

Ocurrió un error... Comuníquese con el equipo de soporte si necesita ayuda.

Ocultar detalles técnicos

```
loaded, processed = load_module_graph(
    .....
File "/usr/lib/python3/dist-packages/odoo/modules/loading.py", line 228, in load_module_graph
load_data(env, idref, mode, kind='data', package=package)
File "/usr/lib/python3/dist-packages/odoo/modules/loading.py", line 72, in load_data
tools.convert_file(env, package.name, filename, idref, mode, noupdate, kind)
File "/usr/lib/python3/dist-packages/odoo/tools/convert.py", line 615, in convert_file
convert_xml_import(env, module, fp, idref, mode, noupdate)
File "/usr/lib/python3/dist-packages/odoo/tools/convert.py", line 686, in convert_xml_import
obj.parse(doc.getroot())
File "/usr/lib/python3/dist-packages/odoo/tools/convert.py", line 601, in parse
self._tag_root(de)
File "/usr/lib/python3/dist-packages/odoo/tools/convert.py", line 557, in _tag_root
raise ParseError('while parsing %s:%s, somewhere inside\n%s' % (
odoo.tools.convert.ParseError: while parsing /mnt/extra-addons/biblioteca_comic/views/biblioteca_comic_views.xml:18,
somewhere inside
<menuitem name="Socios" id="biblioteca_comic_menu_socio" parent="biblioteca_base_menu"
action="biblioteca_comic_actions_socios"/>

The above server error caused the following client error:
RPC_ERROR: Odoo Server Error
RPC_ERROR
at makeErrorFromResponse (http://localhost:8069/web/assets/debug/web.assets_web.js:29920:19)
at XMLHttpRequest.<anonymous> (http://localhost:8069/web/assets/debug/web.assets_web.js:29974:27)
```

Cerrar

4.2 Gestión de préstamos

Para crear un apartado para gestionar los préstamos vamos a seguir más o menos los mismos pasos. Vamos primero a crear un nuevo modelo "biblioteca prestamos.py". Hay que tener en cuenta que cada cómic puede tener varios ejemplares para préstamos por lo que la relación va a ser de varios a uno "Many2One".

Para el archivo modelo va a tener la siguiente estructura, tiene que tener los atributos que nos interesen en este caso necesitamos: el socio al que está prestado, el código del ejemplo prestado y la fecha del inicio y final del préstamo.

```
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields, api
3  from datetime import timedelta
4  from odoo.exceptions import ValidationError
5
6  #Definir Modelo Biblioteca comic
7  class BibliotecaPrestamos(models.Model):
8
9      #Nombre y descripción del modelo
10     _name = 'biblioteca.prestamos'
11     _description = 'Ejemplares de los prestamos'
12
13     ejemplar_id = fields.Many2one('biblioteca.comic', string='Comic', required=True)
14     codigo = fields.Char('Ejemplar código', required=True)
15
16     estado = fields.Selection(
17         [('prestado', 'No disponible'),
18          ('disponible', 'Disponible'),
19          ('perdido', 'Perdido')],
20         string='Estado', default="disponible")
```

```

21 id_socio = fields.Many2one('biblioteca.socio', string='Socio', required=True)
22
23
24 fecha_inicio = fields.Date(string='Fecha de inicio del prestamo')
25 fecha_final = fields.Date(string='Fecha de final del prestamo')
26
27 _rec_name = 'codigo'
28
29 @api.constrains('fecha_inicio')
30 def _check_release_inicio(self):
31     for record in self:
32         #Comprobar de cada registro haya una fecha de inicio y que la fecha no sea
33         #posterior a la actual.
34         if record.fecha_inicio and record.fecha_inicio > fields.Date.today():
35             raise models.ValidationError('La fecha de inicio del prestamo debe ser
36             anterior a la actual')
37
38 @api.constrains('fecha_final')
39 def _check_release_final(self):
40     for record in self:
41         #Comprobar de cada registro haya una fecha final y que la fecha tiene que
42         #ser posterior a la fecha actual.
43         if record.fecha_final and record.fecha_final < fields.Date.today():
44             #Si procede, lanzamos una excepcion
45             raise models.ValidationError('La fecha de final del prestamo debe ser
46             posterior a la actual')

```

Ahora tenemos que crear la vista del modelo que acabamos de crear.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3
4 <!-- Definimos como mostramos la vista en el modelo -->
5 <record id='biblioteca_prestamos_action' model='ir.actions.act_window'>
6     <field name="name">Ejemplares</field>
7     <!-- Indicamos a que modelo aplica -->
8     <field name="res_model">biblioteca.prestamos</field>
9     <!-- Indicamos que los socios pueden verse en list para el listado,
10     y en form para la creacion/edicion -->
11     <field name="view_mode">list,form</field>
12 </record>
13
14 <!-- Definir de la Vista formulario -->
15 <record id="biblioteca_prestamo_view_form" model="ir.ui.view">
16     <field name="name">Formulario de Prestamo de ejemplares</field>
17     <field name="model">biblioteca.prestamos</field>
18     <field name="arch" type="xml">
19         <form>
20             <group>
21                 <group>
22                     <!-- Atributos del ejemplar -->
23                     <field name="ejemplar_id"/>

```

```

24         <field name="codigo"/>
25         <field name="estado"/>
26     </group>
27     <group>
28         <!-- Atributos para el prestamo -->
29         <field name="id_socio"/>
30         <field name="fecha_inicio"/>
31         <field name="fecha_final"/>
32     </group>
33 </group>
34 </form>
35 </field>
36 </record>
37
38 <!-- Definir de la vista list -->
39 <record id="biblioteca_prestamo_view_list" model="ir.ui.view">
40     <field name="name">Lista de Prestamos de la Biblioteca</field>
41     <field name="model">biblioteca.prestamos</field>
42     <field name="arch" type="xml">
43         <list>
44             <field name="codigo"/>
45             <field name="ejemplar_id"/>
46             <field name="id_socio"/>
47             <field name="fecha_inicio"/>
48             <field name="fecha_final"/>
49         </list>
50     </field>
51 </record>
52 </odoo>

```

La vista la tenemos que añadir a nuestro archivo "manifest".

```

1  # -*- coding: utf-8 -*-
2  {
3      'name': "Biblioteca comics",
4      'summary': '"Biblioteca de comics"',
5      'description': '"Gestion de ejemplares y socios"',
6      'author': "Andrea Sofia Pais Dos Santos",
7      'application': True,
8      'category': 'Productivity',
9      'version': '0.1',
10     'depends': ['base'],
11     'data': [
12         'security/ir.model.access.csv',
13         'security/groups.xml',
14         'views/biblioteca_socio.xml', # ponemos las nuevas vistas que vamos creando
15         'views/biblioteca_prestamos.xml', # la nueva vista de los ejemplares y
16         'views/biblioteca_comic_views.xml',
17     ],
18 }

```

Y también tenemos que hacer que esa vista aparezca para que puedas acceder a través del menú principal. Por lo que en la vista principal del módulo vamos a añadir otro item

al menú:

```
1 # .....
2 <menuitem name="Mi biblioteca" id="biblioteca_base_menu" />
3 <menuitem name="Comics" id="biblioteca_comic_menu" parent="biblioteca_base_menu"
4   action="biblioteca_comic_action"/>
5 <!-- Creamos un nuevo apartado para la vista de Socios-->
6 <menuitem name="Socios" id="biblioteca_comic_menu_socio" parent="biblioteca_base_menu"
7   action="biblioteca_comic_action_socios"/>
8 <!-- Creamos un nuevo apartado para la vista de los ejemplares-->
9 <menuitem name="Ejemplares" id="biblioteca_prestamos_menu" parent="
10   biblioteca_base_menu" action="biblioteca_prestamos_action"/>
11 # .....
```

Además tenemos que importa el modelo en el archivo "init" de la carpeta modelos:

```
1 # -*- coding: utf-8 -*-
2
3 from . import biblioteca_comic
4 from . import biblioteca_socio
5 from . import biblioteca_prestamos
```

4.3 Restricciones

Y por último, vamos a tener que añadir la última línea que son los permisos en el archivo "ir.model.access.csv":

```
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_biblioteca_comic,biblioteca.comic.access,model_biblioteca_comic,base.group_user
3   ,1,1,1,1
4 access_biblioteca_socio,biblioteca.socio.access,model_biblioteca_socio,base.group_user
5   ,1,1,1,1
6 access_biblioteca_prestamos,biblioteca.prestamos.access,model_biblioteca_prestamos,
7   base.group_user,1,1,1,1
```

Al rematar, si todo está correcto, actualizamos la lista de aplicaciones y volvemos a actualizar el módulo. Para ver que funciona voy a crear un libro de ejemplo y con el socio que hemos creado anteriormente podemos realizar un préstamo.

Nuevo

Biblioteca de Comics

Alas de Sangre

1 / 1

<

>

Archivar Comics

Titulo	Alas de Sangre	Precio	36,38
Autores	Rebecca Yarros X	Fecha publicación	05/08/2023
Estado	No disponible	Portada Comic	
Numero de páginas ?	736		
Activo	☒	Valoración media lectores	4,7000

Descripción Es una novela de fantasía juvenil y romántica sobre Violet Sorrengail, una joven frágil destinada a los escribas pero forzada por su madre, la comandante, a ingresar al peligroso Colegio de Guerra de Basgiath para convertirse en jinete de dragones, donde debe sobrevivir contra otros cadetes que la quieren muerta y forjar un vínculo con un dragón letal, mientras descubre un oscuro secreto sobre su reino y desarrolla una relación prohibida con el poderoso cadete Xaden Riorson.

Al intentar realizar el préstamo vemos como no nos deja por ejemplo poner que el fin del préstamo tiene que ser días posteriores al día actual.

Mi biblioteca

Comics

Socios

Ejemplares

Nuevo

Ejemplares

Nuevo

Comic	Alas de Sangre	Socio	Andrea Sofia
Ejemplar código	libro01	Fecha de inicio del préstamo	18/12/2025
Estado	Disponible	Fecha de final del préstamo	01/12/2025

¡Ups!

La fecha de final del préstamo debe ser posterior a la actual

Permanecer aquí

Descartar cambios

Y aquí vemos donde iría el listado de todos los préstamos con sus atributos: el código, el nombre del libro, el socio que pide el préstamo y las fechas del préstamo.

Mi biblioteca Comics Socios Ejemplares My Company A					
Nuevo Ejemplares ⚙		Q Buscar...		1-1 / 1 < >	
☐	Ejemplar código	Comic	Socio	Fecha de ...	Fecha de ...
☐	libro01	Alas de Sangre	Andrea Sofía	08/12/2025	18/12/2025

4.4 Errores encontrados

Los errores encontrados siguen siendo constantemente errores al escribir rápido, olvidarme el nombre de los atributos y escribirlos distintos, poner palabras en singular cuando tendría que ser plural, etc.

5 Actividad 03 - Módulo Hospital

Para esta actividad hay que realizar más o menos los mismos pasos que en el anterior cambiando los tipos de relación entre entidades. Hay que crear 3 modelos con sus respectivos atributos y tener en cuenta las relaciones. Voy a crear la carpeta para el nuevo módulo con los archivos base que vamos a ir rellenando a medida que se hacen los apartados.

5.1 Modelo Paciente

Primero vamos a crear el modelo de pacientes empezando a crear el archivo modelo y su respectiva vista con los atributos que nos pide en este caso: nombre, apellidos y síntomas.

```

1  # -*- coding: utf-8 -*-
2  from datetime import timedelta
3
4  from odoo import models, fields, api
5  from odoo.exceptions import ValidationError
6
7
8  # Modelo base, creado como modelo abstracto
9  # Este modelo lo heredarara el modelo HospitalPaciente
10 # Y se ha creado puramente con fin didactico para ver herencia entre modelos
11 class BaseArchive(models.AbstractModel):
12     #Nombre y descripcion del modelo

```

```

13     _name = 'base.archive'
14     _description = 'Fichero abstracto'
15
16     #Introduce el atributo "Activo"
17     activo = fields.Boolean(default=True)
18
19     #Introduce metodo "archivar" que invierte el estado de "activo"
20     #Recordamos se recibe "self" como el modelo entero y no como un registro,
21     #por ese motivo debemos iterar
22     def archivar(self):
23         #Por cada registro del modelo
24         for record in self:
25             #Cambiamos el valor de activo a su version negada
26             record.activo = not record.activo
27
28
29 #Definir Modelo Biblioteca comic
30 class HospitalPaciente(models.Model):
31
32     #Nombre y descripcion del modelo
33     _name = 'hospital.paciente'
34     #Hereda de "base.archive"
35     _inherit = ['base.archive']
36
37     _description = 'Paciente del Hospital'
38
39     #ATRIBUTOS
40
41     #Indicamos que atributo sera el que se usara para mostrar nombre.
42     #Por defecto es "name", pero si no hay un atributo que se llama name, aqui lo
43     #indicamos
44     #Aqui indicamos que se use el atributo "nombre"
45     _rec_name = 'nombre'
46     #Atributo nombre
47     nombre = fields.Char('Nombre', required=True)
48     apellidos = fields.Char('Apellidos', required=True)
49     sintomas = fields.Html('Sintomas', required=True)

```

Ahora creamos la vista donde vamos a poder rellenar el formulario del paciente, listarlo y buscar.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3
4     <!-- Definimos como mostramos la vista en el modelo -->
5     <record id='gestion_hospital.hospital_paciente_action' model='ir.actions.
6         act_window'>
7         <field name="name">Pacientes del hospital</field>
8         <!-- Indicamos a que modelo aplica -->
9         <field name="res_model">hospital.paciente</field>
10        <!-- Indicamos que los pacientes pueden verse en list para el listado,
11        y en form para la creacion/edicion -->
12        <field name="view_mode">list,form</field>
13    </record>
14
15    <!-- Definir de la Vista formulario -->
16    <record id="gestion_hospital.hospital_paciente_view_form" model="ir.ui.view">

```

```

16 <field name="name">Formulario de los pacientes del hospital</field>
17 <field name="model">hospital.paciente</field>
18 <field name="arch" type="xml">
19     <form>
20         <!-- Creamos un boton "archivar".
21             Al llamarse "archivar" se encarga de llamar al metodo "archivar"
22             del modelo del que hereda "hospital.paciente".
23             Este metodo archivar cambie el estado del atributo del modelo "
24                 activo"
25         -->
26         <header>
27             <button type="object" name="archivar" string="Archivar Pacientes"
28                 />
29         </header>
30         <group>
31             <group>
32                 <field name="nombre"/>
33                 <field name="apellidos"/>
34             </group>
35         </group>
36         <group>
37             <field name="sintomas" widget="html"/>
38         </group>
39     </form>
40 </field>
41 </record>
42
43 <!-- Definir de la vista list -->
44 <record id="gestion_hospital.hospital_paciente_view_list" model="ir.ui.view">
45     <field name="name">Lista de los pacientes del hospital</field>
46     <field name="model">hospital.paciente</field>
47     <field name="arch" type="xml">
48         <list>
49             <field name="nombre"/>
50             <field name="apellidos"/>
51             <field name="sintomas" widget="html"/>
52         </list>
53     </field>
54 </record>
55
56 <!-- Definir de la vista busqueda-->
57 <record id="gestion_hospital.hospital_paciente_view_search" model="ir.ui.view">
58     <field name="name">Busqueda de pacientes del hospital</field>
59     <field name="model">hospital.paciente</field>
60     <field name="arch" type="xml">
61         <search>
62             <field name="nombre"/>
63             <field name="apellidos"/>
64         </search>
65     </field>
66 </record>
67 </odoo>

```

Después de haber creado estos archivos tenemos que importarlo en el archivo "init" de la carpeta models.

```

1 # -*- coding: utf-8 -*-

```

```

2
3 from . import hospital_paciente

```

También añadir en el archivo principal "manifest" la vista que hemos creado.

```

1 # -*- coding: utf-8 -*-
2 {
3     'name': "Modulo Hospital",
4     'summary': ""Gestion de un Hospital"",
5     'description': ""Gestion de pacientes, medicos y consultas"",
6     'author': "Andrea Sofia Pais Dos Santos",
7     'application': True,
8     'category': 'Productivity',
9     'version': '0.1',
10    'depends': ['base'],
11    'data': [
12        'security/ir.model.access.csv',
13        'security/groups.xml',
14        'views/hospital_paciente.xml', # ponemos las nuevas vistas que vamos creando
15    ],
16 }

```

Y ya por último darle los permisos al archivo "ir.model.access.csv":

```

1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_hospital_paciente,hospital.paciente.access,model_hospital_paciente,base.
   group_user,1,1,1,1

```

Vamos a crear un archivo que se llama "hospital menu.xml" que se va a encargar de generar los apartados del menú superior y va a tener el siguiente contenido:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <menuitem name="Hospital Central" id="gestion_hospital.hospital_base_menu" />
4     <menuitem name="Pacientes" id="gestion_hospital.hospital_paciente_menu" parent="
      gestion_hospital.hospital_base_menu" action="gestion_hospital.
      hospital_paciente_action"/>
5     <!-- luego vamos a poner los siguientes apartados que ya los dejo ahi abajo -->
6     <menuitem name="Medicos" id="gestion_hospital.hospital_medico_menu" parent="
      gestion_hospital.hospital_base_menu" action="gestion_hospital.
      hospital_medico_action"/>
7     <menuitem name="Consultas" id="gestion_hospital.hospital_consulta_menu" parent="
      gestion_hospital.hospital_base_menu" action="gestion_hospital.
      hospital_consulta_action"/>
8 </odoo>

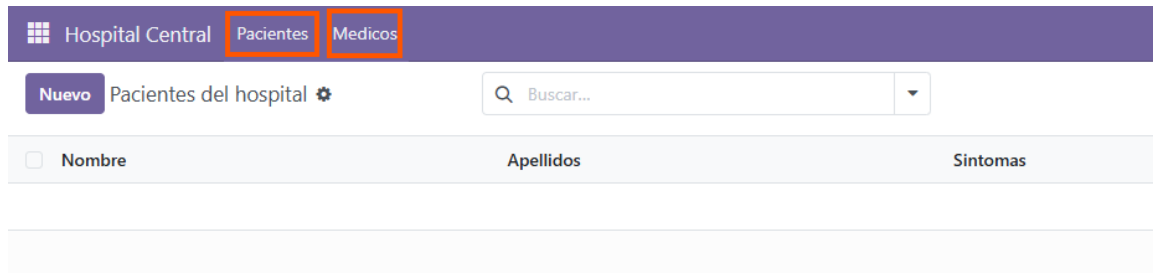
```

Este archivo también hay que añadirlo al archivo "manifest".

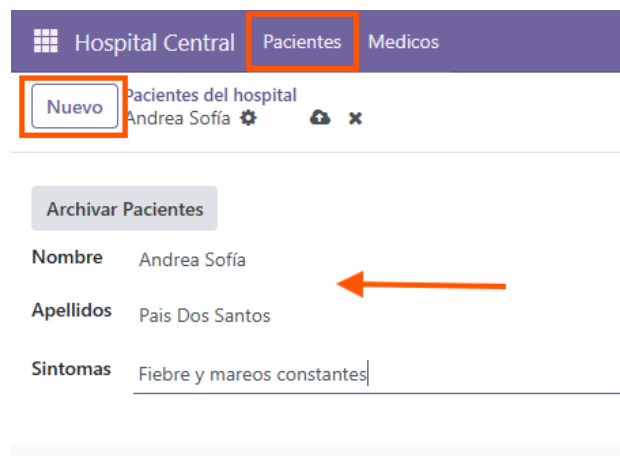
Si todo está correcto, actualizamos la lista de las aplicaciones y buscamos el nuevo creado y lo activamos.



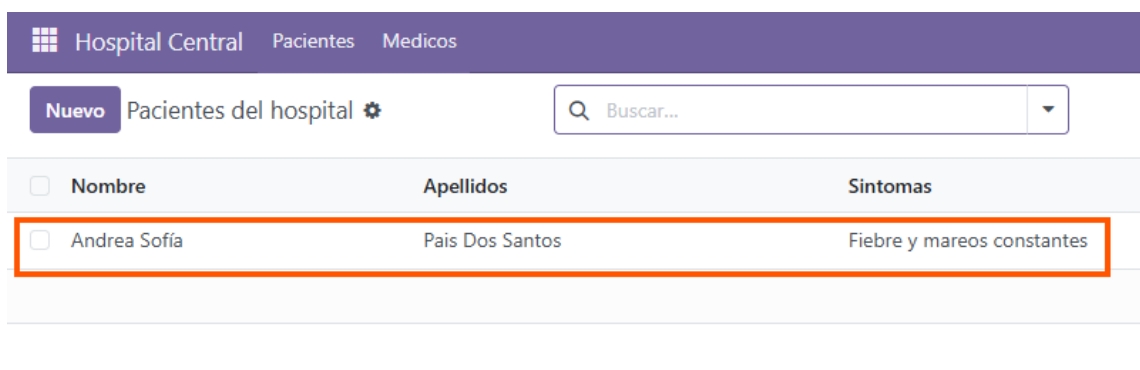
Y al acceder al módulo vemos que se creó la vista pacientes y ya de paso hice la vista médico ya que es prácticamente los mismos pasos, archivos y contenido.



Para ver su correcto funcionamiento vamos a crear un paciente nuevo:



Y como se ve listado ese paciente creado.



5.2 Modelo Médico

Por otro lado tenemos el modelo médico que para crear el modelo y la vista es lo mismo que para realizar el de paciente pero con diferentes atributos, en este caso tenemos que

implementar: nombre, apellidos y número de colegiado. Para el archivo model vamos a tener la siguiente estructura con los atributos anteriores:

```
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields, api
3
4  #Definimos modelo Biblioteca socio
5  class HospitalMedico(models.Model):
6
7      #Nombre y descripcion del modelo
8      _name = 'hospital.medico'
9      #Hereda de "base.archive"
10     _inherit = ['base.archive']
11     _description = 'Medicos del hospital'
12
13     #ATRIBUTOS
14
15     nombre = fields.Char('Nombre', required=True)
16     apellidos = fields.Char('Apellidos', required=True)
17     num_colegiado = fields.Char('Numero colegiado', required=True)
18     #Indicamos que atributo sera el que se usara para mostrar nombre.
19     #Aqui indicamos que se use el atributo "nombre"
20     _rec_name = 'nombre'
21
22     _sql_constraints = [
23         ('num_colegiado_uniq', 'UNIQUE(num_colegiado)', 'El identificador ya existe')
24     ]
```

Ahora creamos la vista donde vamos a poder rellenar el formulario del médico, listarlo y buscar.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3
4      <!-- Definimos como mostramos la vista en el modelo -->
5      <record id='gestion_hospital.hospital_medico_action' model='ir.actions.act_window'
6          >
7          <field name="name">Medico</field>
8          <!-- Indicamos a que modelo aplica -->
9          <field name="res_model">hospital.medico</field>
10         <!-- Indicamos que los medicos pueden verse en list para el listado,
11             y en form para la creacion/edicion -->
12         <field name="view_mode">list,form</field>
13     </record>
14
15     <!-- Definirde la Vista formulario -->
16     <record id="gestion_hospital.hospital_medico_view_form" model="ir.ui.view">
17         <field name="name">Formulario de los medicos del hospital</field>
18         <field name="model">hospital.medico</field>
19         <field name="arch" type="xml">
20             <form>
21                 <!-- Creamos un boton "archivar".
22                     Este metodo archivar cambie el estado del atributo del modelo "
23                     activo"
24                 -->
25             </form>
26         </field>
27     </record>
28 </odoo>
```



```

24         <button type="object" name="archivar" string="Archivar Medicos"/>
25     </header>
26     <group>
27         <group>
28             <!-- Atributos -->
29             <field name="nombre"/>
30             <field name="apellidos"/>
31             <field name="num_colegiado"/>
32         </group>
33     </group>
34 </form>
35 </field>
36 </record>
37
38 <!-- Definir de la vista list -->
39 <record id="gestion_hospital.hospital_medico_view_list" model="ir.ui.view">
40     <field name="name">Lista de medicos del hospital</field>
41     <field name="model">hospital.medico</field>
42     <field name="arch" type="xml">
43         <list>
44             <field name="nombre"/>
45             <field name="apellidos"/>
46             <field name="num_colegiado"/>
47         </list>
48     </field>
49 </record>
50
51 <!-- Definir de la vista busqueda-->
52 <record id="gestion_hospital.hospital_medico_view_search" model="ir.ui.view">
53     <field name="name">Busqueda de los Medicos</field>
54     <field name="model">hospital.medico</field>
55     <field name="arch" type="xml">
56         <search>
57             <field name="nombre"/>
58             <field name="apellidos"/>
59             <field name="num_colegiado"/>
60         </search>
61     </field>
62 </record>
63 </odoo>

```

Después de haber creado estos archivos tenemos que importarlo en el archivo "init" de la carpeta models.

```

1 # -*- coding: utf-8 -*-
2
3 from . import hospital_medico
4 from . import hospital_paciente

```

También añadir en el archivo principal "manifest" la vista que hemos creado.

```

1 # -*- coding: utf-8 -*-
2 {
3     'name': "Modulo Hospital",
4     'summary': ""Gestion de un Hospital"",
5     'description': ""Gestion de pacientes, medicos y consultas"",

```

```

6      'author': "Andrea Sofia Pais Dos Santos",
7      'application': True,
8      'category': 'Productivity',
9      'version': '0.1',
10     'depends': ['base'],
11     'data': [
12         'security/ir.model.access.csv',
13         'security/groups.xml',
14         'views/hospital_medico.xml', # ponemos las nuevas vistas que vamos creando
15         'views/hospital_paciente.xml', # y ahora la vista del modelo medico
16     ],
17 }

```

Y ya por último darle los permisos al archivo "ir.model.access.csv":

```

1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_hospital_paciente,hospital.paciente.access,model_hospital_paciente,base.
  group_user,1,1,1,1
3 access_hospital_medico,hospital.medico.access,model_hospital_medico,base.group_user
  ,1,1,1,1

```

Si todo está correcto, actualizamos la lista de las aplicaciones y actualizamos también el módulo. Al haberlos creado al mismo tiempo ya habíamos visto que el módulo y las dos vistas ahora hay que comprobar que se puede crear tanto pacientes como médicos. Vamos a crear un médico de ejemplo para ver que funciona correctamente:

The screenshot shows the 'Nuevo Medico' form in the 'Hospital Central' application. The form is titled 'Medico Nuevo' and has a 'Nuevo' button. Below the title is an 'Archivar Medicos' button. The form contains three fields: 'Nombre' with the value 'Daniel', 'Apellidos' with the value 'Dos Santos Alonso', and 'Numero colegiado' with the value '22431234123'. An orange arrow points to the 'Apellidos' field.

Y vemos como se lista ese médico creado.

The screenshot shows the 'Medicos' list view in the 'Hospital Central' application. The table has columns for 'Nombre', 'Apellidos', and 'Numero colegiado'. There is one record: Daniel Dos Santos Alonso with ID 22431234123. An orange arrow points to the search bar, and an orange box highlights the first row.

	Nombre	Apellidos	Numero colegiado
☐	Daniel	Dos Santos Alonso	22431234123

5.3 Modelo Consulta

Para crear el modelo Consulta vamos a seguir los mismos pasos que los apartados anteriores, solo hay que tener en cuenta que en esta vista vamos a tener en cuenta la relación entre los Médicos y los Pacientes. En este caso, los médicos pueden realizar consultas a varios clientes y varios clientes pueden tener consultas con varios médicos.

Primero vamos a crear el archivo modelo que va a tener la siguiente estructura con los atributos: identificador y nombre del médico, identificador y nombre del paciente, síntomas y la fecha de la consulta.

```
1  # -*- coding: utf-8 -*-
2  from odoo import models, fields, api
3
4  #Definimos modelo hospital consulta
5  class HospitalConsulta(models.Model):
6
7      #Nombre y descripcion del modelo
8      _name = 'hospital.consulta'
9      #Hereda de "base.archive"
10     _inherit = ['base.archive']
11     _description = 'Consultas del hospital'
12
13     #ATRIBUTOS
14     id_medico = fields.Many2one('hospital.medico', string='Medico', required=True)
15     id_paciente = fields.Many2one('hospital.paciente', string='Paciente', required=True)
16     nombre_medico = fields.Char(string='Medico', related='id_medico.nombre', store=True)
17     nombre_paciente = fields.Char(string='Paciente', related='id_paciente.nombre', store=True)
18
19     sintoma = fields.Text(string='Síntomas', required=True)
20     fecha = fields.Datetime(string='Fecha de la consulta', default=fields.Datetime.now, required=True)
21
22     _rec_name = 'nombre_medico'
```

Creamos la vista del modelo Consulta con la siguiente estructura y los atributos anteriores creados:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3
4      <!-- Definimos como mostramos la vista en el modelo -->
5      <record id='gestion_hospital.hospital_consulta_action' model='ir.actions.act_window'>
6          <field name="name">Consultas</field>
7          <!-- Indicamos a que modelo aplica -->
8          <field name="res_model">hospital.consulta</field>
9          <!-- Indicamos que los socios pueden verse en list para el listado, y en form para la creacion/edicion -->
10         <field name="view_mode">list,form</field>
11     </record>
12
13     <!-- Definir de la Vista formulario -->
14     <record id="gestion_hospital.hospital_consulta_view_form" model="ir.ui.view">
```

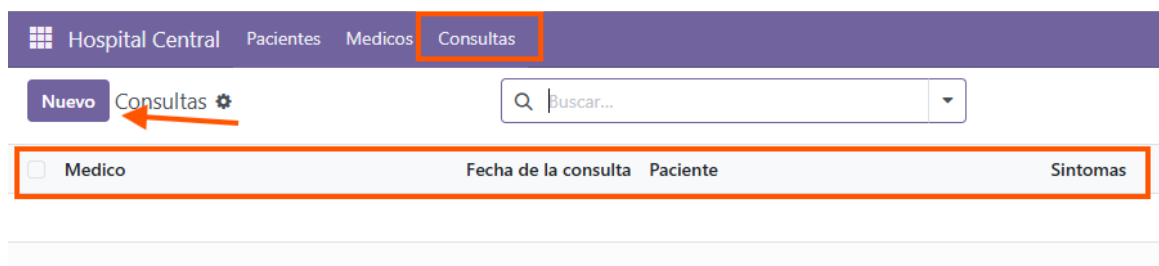
```

16     <field name="name">Formulario de consulta</field>
17     <field name="model">hospital.consulta</field>
18     <field name="arch" type="xml">
19         <form>
20             <group>
21                 <group>
22                     <!-- Atributos del medico -->
23                     <field name="id_medico"/>
24                     <field name="nombre_medico"/>
25                     <field name="fecha"/>
26                 </group>
27                 <group>
28                     <!-- Atributos del paciente -->
29                     <field name="id_paciente"/>
30                     <field name="nombre_paciente"/>
31                     <field name="sintoma"/>
32                 </group>
33             </group>
34         </form>
35     </field>
36 </record>
37
38 <!-- Definir de la vista list -->
39 <record id="gestion_hospital.hospital_consulta_view_list" model="ir.ui.view">
40     <field name="name">Lista de consultas</field>
41     <field name="model">hospital.consulta</field>
42     <field name="arch" type="xml">
43         <list>
44             <field name="nombre_medico"/>
45             <field name="fecha"/>
46             <field name="nombre_paciente"/>
47             <field name="sintoma"/>
48         </list>
49     </field>
50 </record>
51 </odoo>

```

Es importante importar el modelo en el archivo "init" de la carpeta modelo, añadir la vista en el "manifest" y en el archivo de seguridad los permisos como en las anteriores vistas.

Si todo está correcto, actualizamos el módulo y deberíamos poder ver el nuevo apartado "Consultas" con sus respectivos atributos.



Ahora vamos a crear un ejemplo donde, un médico tiene varias consultas con pacientes distintos. Aquí un ejemplo visual de como se crea una consulta.

Y aquí vemos que el médico Daniel tiene dos consultas con dos pacientes diferentes en distintas fechas con sus respectivos síntomas. Y pasaría lo mismo si un paciente en este caso Andrea quiere tener varias citas con médicos diferentes.

Hospital Central Pacientes Medicos Consultas				
Nuevo Consultas		Q Buscar...		
☐ Medico	Fecha de la consulta	Paciente	Síntomas	
☐ Daniel	10/12/2025 18:20:59	Andrea Sofia	Fiebre y mareos constantes	
☐ Daniel	11/12/2025 18:21:55	Xade	Dolor de cabeza	
☐ María	13/12/2025 19:21:33	Andrea Sofia	Extracción de sangre	

5.4 Errores encontrados

Me daban algunos errores al crear la nueva vista Consulta, porque yo antes creaba las vistas del menú en la vista paciente que es la principal. Busqué y recomendaban crear un nuevo archivo exclusivamente para el menú del módulo. Creé y separé los menús en otro apartado y ya solucioné los problemas que tenía.

6 Actividad 04 - Módulo Colegio

El objetivo de la última actividad es crear un módulo de Odoo con cuatro modelos:

- Modelo Ciclo Formativo que puede estar asociado a uno o más de un módulo.
- Modelo Módulo que pertenece a un ciclo formativo, tiene que tener alumnos matriculados y tener un profesor que imparta el módulo.
- Modelo Alumno con sus atributos y se relaciona con los módulos a los que se matricule.
- Modelo Profesor con sus atributos y se relaciona con los módulos a los que se matricule.

Vamos a empezar a crear una carpeta para el nuevo módulo con los archivos base que vamos a ir completando. Ahora vamos a tener en cuenta los atributos que vamos a crear para cada modelo y que relación van a tener:

En el modelo "Ciclo Formativo" vamos a tener el atributo "id modulos" en el que la relación sea (One2many) por lo que 1 Ciclo puede tener varios Módulos (1:N).

En el modelo "Módulos" vamos a tener el atributo "id ciclos" en la que la relación es (One2many) por lo que a 1 Módulo pertenece a un Ciclo (1:N). También necesitaremos el atributo "id profesor" que la relación tiene que ser (Many2one) en el que ese Módulo es impartido por un único profesor (N:1). Y el "id alumno" que tendrá una relación (Many2many) en el que un módulo tiene muchos alumnos y el alumno puede estar en varios módulos matriculados (N:M).

Después en el modelo "Alumno" vamos a necesitar tener por un lado el atributo "id modulos" con una relación de (Many2many) donde el alumno puede estar en muchos módulos y un módulo tiene varios alumnos (N:M).

Y en el modelo "Profesor" necesitamos el atributo "id modulos" con una relación de (One2many) donde el profesor imparte uno o muchos módulos (1:N).

Tras ver que hay muchos atributos que se relacionan entre modelos es más cómodo realizar ya todos las vistas y modelos al mismo tiempo porque sino no van a funcionar la carga de la aplicación en Odoo yendo vista a vista para enseñar. Por lo que decidí crearlo todo de manera conjunta y ver que todo funciona al final de crear todas las vistas para ver que las relaciones se hacen correctamente.

6.1 Modelo Ciclo Formativo

Vamos a empezar creando el archivo módulo "Ciclo" con la siguiente estructura y los atributos que mencionamos anteriormente más los que creamos necesarios para el ciclo.

```
1 from odoo import models, fields, api
2
3 class CicloFormativo(models.Model):
4     _name = 'colegio.ciclo'
5     _description = 'Ciclo Formativo'
6
7     nombre = fields.Char(string='Nombre del ciclo',required=True)
8     codigo_ciclo = fields.Char(string='Codigo',required=True)
9     descripcion = fields.Html(string='Descripcion')
10    id_modulos = fields.One2many('colegio.modulo','id_ciclo',string='Modulos')
```

Para la vista de ciclo nos interesa ver el formulario con los atributos que creamos y la opción de añadir los módulos que va a contener ese ciclo.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Accion para Ciclos -->
4     <record id='gestion_colegio.colegio_ciclo_action' model='ir.actions.act_window'>
5         <field name="name">Ciclos Formativos</field>
6         <field name="res_model">colegio.ciclo</field>
7         <field name="view_mode">list,form</field>
```

```

8     </record>
9
10    <!-- Vista lista de Ciclos -->
11    <record id="gestion_colegio.colegio_ciclo_view_list" model="ir.ui.view">
12        <field name="name">Lista de Ciclos Formativos</field>
13        <field name="model">colegio.ciclo</field>
14        <field name="arch" type="xml">
15            <list>
16                <field name="nombre"/>
17                <field name="codigo_ciclo"/>
18                <field name="descripcion"/>
19            </list>
20        </field>
21    </record>
22
23    <!-- Vista formulario de Ciclos -->
24    <record id="gestion_colegio.colegio_ciclo_view_form" model="ir.ui.view">
25        <field name="name">Formulario de Ciclo Formativo</field>
26        <field name="model">colegio.ciclo</field>
27        <field name="arch" type="xml">
28            <form>
29                <sheet>
30                    <group>
31                        <group>
32                            <field name="nombre"/>
33                            <field name="codigo_ciclo"/>
34                        </group>
35                        <group>
36                            <field name="descripcion" widget="html"/>
37                        </group>
38                    </group>
39                    <notebook>
40                        <page string="Modulos">
41                            <field name="id_modulos">
42                                <list>
43                                    <field name="nombre"/>
44                                    <field name="codigo_modulo"/>
45                                    <field name="horas"/>
46                                    <field name="creditos"/>
47                                    <field name="curso"/>
48                                    <field name="id_profesor" widget="many2one"/>
49                                </list>
50                            </field>
51                        </page>
52                    </notebook>
53                </sheet>
54            </form>
55        </field>
56    </record>
57
58    <!-- Vista busqueda de Ciclos -->
59    <record id="gestion_colegio.colegio_ciclo_view_search" model="ir.ui.view">
60        <field name="name">Busqueda de Ciclos</field>
61        <field name="model">colegio.ciclo</field>
62        <field name="arch" type="xml">
63            <search>
64                <field name="nombre"/>
65                <field name="codigo_ciclo"/>

```

```

66         <field name="descripcion"/>
67     </search>
68 </field>
69 </record>
70 </odoo>

```

Luego de tener la vista la añadimos al archivo "manifest", importamos el modelo al archivo "init" de la carpeta "models" y añadimos los permisos que consideramos por ahora, que posteriormente lo vamos a tener que cambiar.

Aquí vemos como ya están todas las vistas, la parte de seguridad(que para el último apartado de la práctica vamos a tener que modificar) y nuestra vista con los menús.

```

1  # -*- coding: utf-8 -*-
2  {
3      'name': "Gestion del Colegio Princesa de Espana",
4      'summary': ""Gestion de ciclos formativos, modulos, alumnos y profesores"",
5      'description': ""Modulo para la gestion completa de un instituto educativo:"",
6      'author': "Andrea Sofia Pais Dos Santos",
7      'application': True,
8      'category': 'Education',
9      'version': '0.1',
10     'depends': ['base'],
11     'data': [
12         # primero seguridad, luego vistas, luego menus
13         'security/group.xml',
14         'security/ir.model.access.csv',
15
16         # Vistas en orden alfabético para claridad
17         'views/colegio_ciclo.xml',
18         'views/colegio_modulo.xml',
19         'views/colegio_alumno.xml',
20         'views/colegio_profesor.xml',
21
22         # Menus al final
23         'views/colegio_menu.xml',
24     ],
25 }

```

Importamos en el archivo "init" todos los modelos que vamos a tener que crear.

```

1  # -*- coding: utf-8 -*-
2
3  from . import colegio_ciclo
4  from . import colegio_modulo
5  from . import colegio_alumno
6  from . import colegio_profesor

```

Y por ahora nuestro archivo "ir.model.access.csv" por ahora vamos a dar los siguientes permisos para que se vean nuestras vistas en Odoo.

```

1  id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2  access_colegio_ciclo_user,access_colegio_ciclo_user,model_colegio_ciclo,
    gestion_colegio.grupo_administrativos,1,1,1,1

```

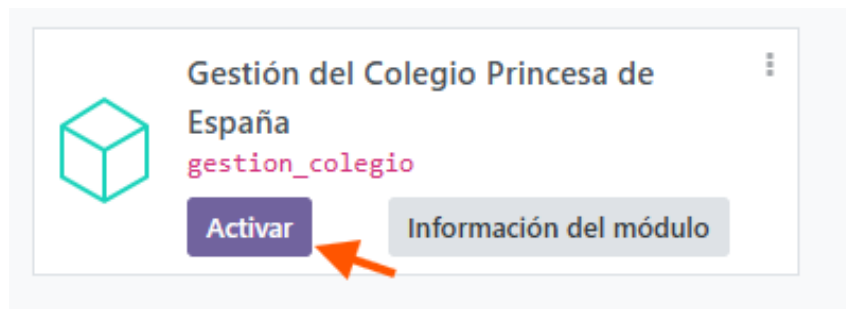


```

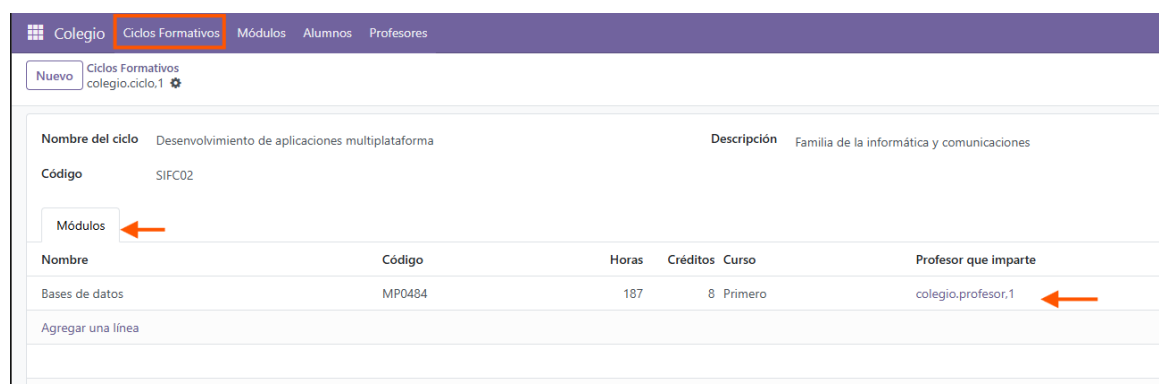
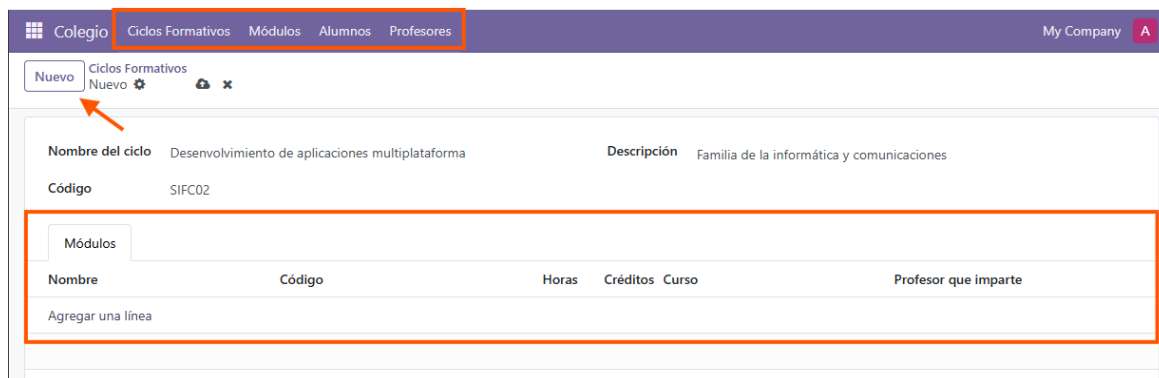
3 access_colegio_modulo_user,access_colegio_modulo_user,model_colegio_modulo,
  gestion_colegio.grupo_administrativos,1,1,1,1
4 access_colegio_alumno_user,access_colegio_alumno_user,model_colegio_alumno,
  gestion_colegio.grupo_administrativos,1,1,1,1
5 access_colegio_profesor_user,access_colegio_profesor_user,model_colegio_profesor,
  gestion_colegio.grupo_administrativos,1,1,1,1

```

Si todo está correcto, levantamos el contenedor de Docker y iniciamos en Odoo con nuestros datos. Actualizamos la lista de aplicaciones y activamos el módulo que acabamos de crear.



Ahora vamos a ver un ejemplo de como se crea un nuevo "Ciclo" con sus atributos y un ejemplo de módulo que creamos luego que sino no nos va a aparecer. Al tener el módulo activamos podemos ver las 4 vistas que teníamos que crear.



Y nos permite acceder a la información de cada uno de los módulos que vayamos añadiendo al ciclo y vemos a que ciclo pertenece y que profesor imparte el módulo.

Abrir: Módulos

Nombre	Bases de datos	Curso	Primero
Código	MP0484	Ciclo Formativo	colegio.ciclo,1
Horas	187	Profesor que imparte	colegio.profesor,1
Créditos	8		

Nombre	Apellidos	DNI del alumno/a	Fecha de ...
Andrea Sofía	Pais Dos Santos	35651502F	11/12/2025
Agregar una línea			

6.2 Modelo Módulo

Para el modelo Módulo vamos a realizar el mismo procedimiento que para el modelo anterior, teniendo en cuenta los atributos que hemos definido con anterioridad con las relaciones que necesitamos y atributos que son relevantes para tener información suficiente de cada módulo como horas, créditos, curso al que pertenece, etc. Por lo que nuestro archivo "colegio modulo.py" va a estar estructurado de la siguiente forma:

```

1 from odoo import models, fields, api
2
3 class ColegioModulo(models.Model):
4     _name = 'colegio.modulo'
5     _description = 'Modulo'
6
7     nombre = fields.Char(string='Nombre',required=True)
8     codigo_modulo = fields.Char(string='Codigo',required=True)
9     horas = fields.Integer(string='Horas')
10    credits = fields.Integer(string='Creditos')
11    curso = fields.Selection([
12        ('1','Primero'),
13        ('2','Segundo')
14    ], string = 'Curso',required=True)
15    # relaciones que se necesitan
16    id_ciclo = fields.Many2one('colegio.ciclo',string='Ciclo Formativo',required=True)
17    id_profesor = fields.Many2one('colegio.profesor',string='Profesor que imparte')
18    id_alumno = fields.Many2many('colegio.alumno',string='Alumnos matriculados')
19
20    # restriccion del codigo unico
21    _sql_constraints = [
22        ('codigo_modulo_uniq','UNIQUE(codigo_modulo,id_ciclo)','El codigo del modulo
23        debe de ser unico')
24    ]

```

Luego creamos nuestra vista para crear, listar y buscar nuestros módulos y con los atributos hemos creado en el archivo modelo y nos interesa saber a que profesor lo va a impartir y a que ciclo y curso pertenece.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Accion para Modulo -->
4     <record id='gestion_colegio.colegio_modulo_action' model='ir.actions.act_window'>
5         <field name="name">Modulos</field>
6         <field name="res_model">colegio.modulo</field>
7         <field name="view_mode">list,form</field>
8     </record>
9
10    <!-- Vista de lista de Modulo -->
11    <record id="gestion_colegio.colegio_modulo_view_list" model="ir.ui.view">
12        <field name="name">Lista de Modulos</field>
13        <field name="model">colegio.modulo</field>
14        <field name="arch" type="xml">
15            <list>
16                <field name="nombre"/>
17                <field name="codigo_modulo"/>
18                <field name="horas"/>
19                <field name="creditos"/>
20                <field name="curso"/>
21                <field name="id_ciclo"/>
22                <field name="id_profesor"/>
23            </list>
24        </field>
25    </record>
26
27    <!-- Vista de formulario de Modulo -->
28    <record id="gestion_colegio.colegio_modulo_view_form" model="ir.ui.view">
29        <field name="name">Formulario de Modulo</field>
30        <field name="model">colegio.modulo</field>
31        <field name="arch" type="xml">
32            <form>
33                <sheet>
34                    <group>
35                        <group>
36                            <field name="nombre"/>
37                            <field name="codigo_modulo"/>
38                            <field name="horas"/>
39                            <field name="creditos"/>
40                        </group>
41                        <group>
42                            <field name="curso"/>
43                            <field name="id_ciclo" widget="many2one"/>
44                            <field name="id_profesor" widget="many2one"/>
45                        </group>
46                    </group>
47                    <field name="id_alumno">
48                        <list>
49                            <field name="nombre"/>
50                            <field name="apellidos"/>
51                            <field name="dni"/>
52                            <field name="fecha_matriculacion"/>
53                        </list>
54                    </field>
55                </sheet>
56            </form>
57        </field>

```

```

58     </record>
59
60     <!-- Vista de busqueda de Modulo -->
61     <record id="gestion_colegio.colegio_modulo_view_search" model="ir.ui.view">
62         <field name="name">Busqueda de modulos</field>
63         <field name="model">colegio.modulo</field>
64         <field name="arch" type="xml">
65             <search>
66                 <field name="nombre"/>
67                 <field name="codigo_modulo"/>
68                 <field name="curso"/>
69                 <field name="id_ciclo"/>
70                 <field name="id_profesor"/>
71             </search>
72         </field>
73     </record>
74 </odoo>

```

Luego hay que hacer lo mismo en toda creación de modelos y vistas, añadirlo al "manifest", importar el modelo al "init" de la carpeta models y añadir los permisos que consideremos. (Ya están completados en el anterior modelo)

Ahora lo que nos interesa es realizar un ejemplo de como creamos un módulo que vemos los atributos del módulo, más a que ciclo pertenece, el profesor que lo imparte y una lista de los alumnos que pertenecen a ese módulo.

Y así se vería la lista de los módulos.

☐ Nombre	Código	Horas	Créditos	Curso	Ciclo Formativo	Profesor que imparte
☐ Bases de datos	MP0484	187	8	Primero	colegio.ciclo,1	colegio.profesor,1

6.3 Modelo Alumno

Para el modelo "Alumno" vamos a seguir los mismos pasos, lo único que cambia es que atributos necesita y los que tienen relación a otros modelos.

El archivo "colegio alumno.py" va a estar estructurado de la siguiente manera en la que también vamos a restringir que el dni solo pueda tener 9 caracteres par evitar posibles errores.

```
1 from odoo import models, fields, api
2 from odoo.exceptions import ValidationError
3
4 class ColegioAlumno(models.Model):
5     _name = 'colegio.alumno'
6     _description = 'Alumno del colegio'
7
8     nombre = fields.Char(string='Nombre',required=True)
9     apellidos = fields.Char(string='Apellidos',required=True)
10    dni = fields.Char(string='DNI del alumno/a',required=True)
11    fecha_nacimiento = fields.Date(string='Fecha de nacimiento',required=True)
12    telefono = fields.Char(string='Telefono del familiar')
13    fecha_matriculacion = fields.Date('Fecha de matriculacion',default=fields.Date.
        today)
14    # relaciones que se necesitan
15    id_ciclo = fields.Many2one('colegio.ciclo',string='Ciclo matriculado')
16    id_modulo = fields.Many2many('colegio.modulo',string='Modulos matriculados')
17
18    # restriccion del dni unico
19    _sql_constraints = [
20        ('dni_uniq', 'UNIQUE(dni)', 'El DNI ya existe')
21    ]
22
23    @api.constrains('dni')
24    def _check_dni(self):
25        for colegio_alumno in self:
26            if colegio_alumno.dni and len(colegio_alumno.dni) != 9:
27                raise ValidationError('El DNI debe tener 9 caracteres')
```

Ahora vamos a crear la vista del alumno para crear, listar y buscarlos. Nos interesa enseñar al que ciclo pertenece el alumno y a que módulo lo vamos a matricular con su respectiva información.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Accion para Alumno -->
4     <record id='gestion_colegio.colegio_alumno_action' model='ir.actions.act_window'>
5         <field name="name">Alumnos del colegio</field>
6         <field name="res_model">colegio.alumno</field>
7         <field name="view_mode">list,form</field>
8     </record>
9
10    <!-- Vista de lista de Alumno -->
11    <record id="gestion_colegio.colegio_alumno_view_list" model="ir.ui.view">
12        <field name="name">Lista de Alumnos</field>
13        <field name="model">colegio.alumno</field>
14        <field name="arch" type="xml">
15            <list>
16                <field name="nombre"/>
17                <field name="apellidos"/>
18                <field name="dni"/>
19                <field name="fecha_nacimiento"/>
```

```

20         <field name="telefono"/>
21         <field name="fecha_matriculacion"/>
22         <field name="id_ciclo"/>
23     </list>
24 </field>
25 </record>
26
27 <!-- Vista de formulario de Alumno -->
28 <record id="gestion_colegio.colegio_alumno_view_form" model="ir.ui.view">
29     <field name="name">Formulario de Alumno</field>
30     <field name="model">colegio.alumno</field>
31     <field name="arch" type="xml">
32         <form>
33             <sheet>
34                 <group>
35                     <group>
36                         <field name="nombre"/>
37                         <field name="apellidos"/>
38                         <field name="dni"/>
39                         <field name="fecha_nacimiento"/>
40                     </group>
41                     <group>
42                         <field name="id_ciclo" widget="many2one"/>
43                         <field name="fecha_matriculacion"/>
44                     </group>
45                 </group>
46                 <field name="id_modulo">
47                     <list>
48                         <field name="nombre"/>
49                         <field name="codigo_modulo"/>
50                         <field name="creditos"/>
51                         <field name="id_profesor"/>
52                     </list>
53                 </field>
54             </sheet>
55         </form>
56     </field>
57 </record>
58
59 <!-- Vista de busqueda de Alumno -->
60 <record id="gestion_colegio.colegio_alumno_view_search" model="ir.ui.view">
61     <field name="name">Busqueda de Alumno</field>
62     <field name="model">colegio.alumno</field>
63     <field name="arch" type="xml">
64         <search>
65             <field name="nombre"/>
66             <field name="apellidos"/>
67             <field name="dni"/>
68             <field name="id_ciclo"/>
69         </search>
70     </field>
71 </record>
72 </odoo>

```

Luego hay que hacer lo mismo en toda creación de modelos y vistas, añadirlo al "manifest", importar el modelo al "init" de la carpeta models y añadir los permisos que consideremos. (Ya están completados en el anterior modelo)

Ahora lo que nos interesa es realizar un ejemplo de como creamos un Alumno con sus atributos, a que ciclo está matriculado y al módulo que se va a matricular.

6.4 Modelo Profesor

Para el modelo "Profesor" vamos a seguir los mismos pasos, lo único que cambia es que atributos que va a necesita y los que tienen relación a otros modelos.

El archivo "colegio profesor.py" va a estar estructurado de la siguiente manera en la que también vamos a restringir que el dni solo pueda tener 9 caracteres par evitar posibles errores.

```

1 from odoo import models, fields, api
2 from odoo.exceptions import ValidationError
3
4 class ColegioProfesor(models.Model):
5     _name = 'colegio.profesor'
6     _description = 'Profesor del colegio'
7
8     nombre = fields.Char(string='Nombre',required=True)
9     apellidos = fields.Char(string='Apellidos',required=True)
10    dni = fields.Char(string='DNI del profesor/a',required=True)
11    email = fields.Char(string='Email del colegio',required=True)
12    telefono = fields.Char(string='Telefono')
13    fecha_contratacion = fields.Date('Fecha de contratacion',default=fields.Date.today
14    )
15    # relaciones que se necesitan
16    id_modulo = fields.One2many('colegio.modulo','id_profesor',string='Modulos que
17    imparte')
18
19    # restriccion del dni unico
20    _sql_constraints = [
21        ('dni_uniq', 'UNIQUE(dni)', 'El DNI ya existe'),
22        ('email_uniq','UNIQUE(email)','El email ya existe')
23    ]
24
25    @api.constrains('dni')
26    def _check_dni(self):
27        for colegio_alumno in self:
28            if colegio_alumno.dni and len(colegio_alumno.dni) != 9:
29                raise ValidationError('El DNI debe tener 9 caracteres')

```

Ahora vamos a crear la vista del profesor para crear, listar y buscarlos. Nos interesa enseñar al módulo que imparte con sus respectivos datos y al ciclo que pertenece ese módulo.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Accion para Profesor -->
4     <record id='gestion_colegio.colegio_profesor_action' model='ir.actions.act_window'
5         >
6         <field name="name">Profesor del colegio</field>
7         <field name="res_model">colegio.profesor</field>
8         <field name="view_mode">list,form</field>
9     </record>
10
11     <!-- Vista de lista de profesor -->
12     <record id="gestion_colegio.colegio_profesor_view_list" model="ir.ui.view">
13         <field name="name">Lista de Profesores</field>
14         <field name="model">colegio.profesor</field>
15         <field name="arch" type="xml">
16             <list>
17                 <field name="nombre"/>
18                 <field name="apellidos"/>
19                 <field name="dni"/>
20                 <field name="email"/>
21                 <field name="telefono"/>
22                 <field name="fecha_contratacion"/>
23             </list>
24         </field>
25     </record>
26
27     <!-- Vista de formulario de Profesor -->
28     <record id="gestion_colegio.colegio_profesor_view_form" model="ir.ui.view">
29         <field name="name">Formulario de Profesor</field>
30         <field name="model">colegio.profesor</field>
31         <field name="arch" type="xml">
32             <form>
33                 <sheet>
34                     <group>
35                         <group>
36                             <field name="nombre"/>
37                             <field name="apellidos"/>
38                             <field name="dni"/>
39                         </group>
40                         <group>
41                             <field name="email"/>
42                             <field name="telefono"/>
43                             <field name="fecha_contratacion"/>
44                         </group>
45                     </group>
46                     <field name="id_modulo">
47                         <list>
48                             <field name="nombre"/>
49                             <field name="codigo_modulo"/>
50                             <field name="creditos"/>
51                             <field name="curso"/>
52                             <field name="id_ciclo"/>
53                         </list>
54                     </field>
55                 </sheet>
56             </form>
57         </field>
58     </record>
59 </odoo>
```



```

53         </field>
54     </sheet>
55 </form>
56 </field>
57 </record>
58
59 <!-- Vista de búsqueda de profesor -->
60 <record id="gestion_colegio.colegio_profesor_view_search" model="ir.ui.view">
61     <field name="name">Busqueda de Profesor</field>
62     <field name="model">colegio.profesor</field>
63     <field name="arch" type="xml">
64         <search>
65             <field name="nombre"/>
66             <field name="apellidos"/>
67             <field name="dni"/>
68             <field name="email"/>
69         </search>
70     </field>
71 </record>
72 </odoo>

```

Luego hay que hacer lo mismo en toda creación de modelos y vistas, añadirlo al "manifest", importar el modelo al "init" de la carpeta models y añadir los permisos que consideremos. (Ya están completados en el anterior modelo)

Ahora lo que nos interesa es realizar un ejemplo de como creamos un Profesor con sus atributos, al módulo que va a impartir.

The screenshot shows the 'Profesores' form in Odoo. The top navigation bar includes 'Colegio', 'Ciclos Formativos', 'Módulos', 'Alumnos', and 'Profesores' (highlighted). The form has a 'Nuevo' button and a 'Nuevo' label. The form fields are:

- Nombre: Daniel
- Apellidos: Dos Santos Alonso
- Email del colegio: danielp@colegioblanco.es
- Teléfono: 669050213
- DNI del profesor/a: 35651539K
- Fecha de contratación: 11/12/2025

Below the form fields is a table with the following columns: Nombre, Código, Créditos, Curso, and Ciclo Formativo. An orange arrow points to the 'Nombre' field in the table.

Ya se realizaron las cuatro modelos y sus pistas teniendo en cuenta las relaciones entre las entidades.

6.5 Configuración de seguridad

El objetivo ahora es tras crear el módulo correctamente con sus vistas vamos a implementar dos roles: el de DIRECTOR (va a poder acceder a todo) y PROFESOR (solo tendrá permisos de lectura sobre las vistas). Para ello vamos a tener que modificar nuestros archivos de seguridad. Empezando por el archivo "group.xml".

En el archivo "group.xml" vamos a crear 2 grupos, el grupo director que es el que va a estar por defecto y el grupo profesor.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <!-- Grupo para Directores (acceso completo) -->
4     <record id="group_director" model="res.groups">
5         <field name="name">Director</field>
6         <field name="category_id" ref="base.module_category_services"/>
7         <!-- Asigna el usuario admin como director inicial -->
8         <field name="users" eval="[(4, ref('base.user_admin'))]" />
9     </record>
10
11     <!-- Grupo para Profesores (solo lectura de profesores) -->
12     <record id="group_profesor" model="res.groups">
13         <field name="name">Profesor</field>
14         <field name="category_id" ref="base.module_category_services"/>
15     </record>
16 </odoo>

```

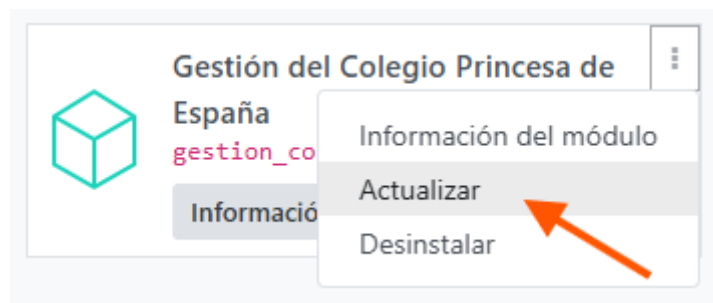
Y ahora en el archivo "ir.model.access.csv" vamos a determinar los permisos que va a tener cada rol en cada vista. Y los usuarios normales no va a poder acceder a nada por lo que el módulo no aparecerá. El director podrá ver todo, modificar, eliminar y crear mientras que el profesor solo podrá ver la vista profesor y no podrá modificar nada.

```

1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 # permisos del grupo director
3 access_colegio_ciclo_director,access_colegio_ciclo_director,model_colegio_ciclo,
4   gestion_colegio.group_director,1,1,1,1
5 access_colegio_modulo_director,access_colegio_modulo_director,model_colegio_modulo,
6   gestion_colegio.group_director,1,1,1,1
7 access_colegio_alumno_director,access_colegio_alumno_director,model_colegio_alumno,
8   gestion_colegio.group_director,1,1,1,1
9 access_colegio_profesor_director,access_colegio_profesor_director,
10   model_colegio_profesor,gestion_colegio.group_director,1,1,1,1
11 # permisos del grupo profesor
12 access_colegio_profesor_profesor,access_colegio_profesor_profesor,
13   model_colegio_profesor,gestion_colegio.group_profesor,1,0,0,0

```

Al realizar los cambios, actualizamos el módulo.



Nos dirigimos a ajustes -> administrar usuarios y más abajo vemos una sección que es la tiene los 2 roles que hemos creado, por defecto vemos que está director puesto que nos permite acceder a todo el módulo.

Mientras que si cambiamos al rol de profesor, solo nos deja ver la vista de profesores.

6.6 Errores encontrados

Los errores siguieron siendo los mismos, errores de sintaxis por usar por ejemplo singular en vez de plural o al revés. Porque al ser la base una "copia" de los anteriores actividades era más fácil no cometer errores, solo había que tener cuidado con hacer bien las relaciones que si están definidas antes se hace más llevadero. Algún error tuve con una relación pero era porque me equivoqué y sin querer la puse al revés.

Las advertencias de Odoo o los logs del contenedor, nos va indicando los errores y nos dan pistas para ver donde nos hemos equivocado.

7 Conclusiones

Al empezar la práctica fue bastante confuso por donde empezar y organizarse de la manera más óptima para realizar correctamente el módulo. Fue todo prueba y error en los 2 primeros ejercicios. Aunque al empezar a realizar las otras actividades se veía un patrón de como es mejor realizar el módulo y como documentarlo de la mejor manera.

Poco a poco, se hacía algo más "ameno" podía reciclar las bases de los modelos y los módulos y a la vez empezaba a ver más fácilmente donde fallaba y más cuando eran errores tontos de palabras mal escritas que fue lo que más rabia me dio de la actividad. He perdido mucho tiempo por olvidarme de como había nombrado atributos en los modelos y luego en las vistas ponía yo otra que pensaba que era esa y no lo era.

En conclusión, fue un trabajo bastante largo pero la verdad es que aprendí bastante en poco tiempo como estructurar los módulos correctamente e ir comprobando que todo funcione correctamente.